Integration of AI-assisted Code Generation in Programming Assessments

Dr Ismahane Cheheb
Department of Computer Science, School of Science and Technology, Nottingham Trent University
Clifton Campus, Nottingham, NG11 8NS, United Kingdom

Introduction

The recent rise of Generative AI popularised via Conversational AI tools such as OpenAI's ChatGPT and Google's Bard has taken the world by storm. It has become apparent that the higher education sector needs to adapt. Several authors have discussed the different challenges that educators and education institutions (Lim et al. 2023; Sullivan, Kelly, and McLaughlan 2023).

However, fewer scholars discussed the issues arising from language models generating code such as Google's Bard and OpenAI's Chat-GPT and how it effects learning in computer science courses. More AI tools that specifically assist in code writing are also appearing such as Github Copilot and Amazon Code Whisperer that are marketed as coding productiveness tools.

Work by Lau and Guo (2023) discusses how computing instructors are planning to adapt to AI coding tools. An important point raised is that students must be prepared for industry jobs where using AI-assisted code writing might become standard.

With the aim of determining a practical approach to embrace AI-assisted code generation tools, this study proposes an evaluation of how students use generative AI tools to refactor code and their ability to critically analyse its output in order to determine code quality and correctness.

Research Question

How might students employ and collaborate with generative AI tools? How can we embed AI in assessments for enhanced student learning?

Objectives

- Evaluate students' critical analysis skills regarding AI-generated code.
- Determine any added value of integrating AI code generation tools in programming assessments.
- Determine best practices of using AI-assisted code generation in summative assessments.

Methodology

This study focuses on learners enrolled in Year 2 Software Engineering DA module. The summative assessment traditionally includes a code refactoring task. Learners are also required to justify their refactoring considerations in a separate document. The piece of code is provided alongside corresponding unit tests.

As part of this phase, the assessment specification has been amended integrating AI. Students are required to:

1) Refactor a given piece of code;
2) Query one of the publicly available generative AI tools to perform an alternative refactoring of the same piece of code.
3) Undertake a comparative analysis, drawing distinctions between their solution and the AI-generated.
4) Articulate which of the AI tool's recommendations they would incorporate in their original solution.

Learners' submissions are then analysed using a qualitative approach that would allow a better understanding of learners' considerations. Analysed data will include both submitted code and written report, putting special emphasis on the following aspects:

- **Critical analysis:** inspection of learners' critical evaluations of the AI-proposed refactoring considerations.
- **Error identification:** assessment of learners' ability to identify mistakes within the AI-generated code
- **Integration proficiency:** evaluation of the effectiveness of integration of AI suggestions in learners' original code.

Analysis and Results (TBC)

It is likely that the responses students get from AI tools will depend on the wording of their query and the AI tool they chose to use. It is also a known fact that AI tools are trained using a variety of online content and code, not all of which uses the best refactoring practices.

Therefore, anticipated results are as follows:

Correctness: AI-generated code may include errors and refer to legacy libraries. Students tend to note those issues and comment on them. However, not all mistakes are spotted, and students may include incorrect code in their own solutions. This may slow down their progress.

Quality: AI's proposed solution may not follow all the best practices but may also include refactoring considerations that would enhance students' original solution.
It is expected that students who are not confident in their programming skills may include low-quality code in their original solution. However, most students should be able to integrate the best refactoring considerations within their original code thus enhancing their experience and providing them with different perspectives that are highly important in the field of software engineering.

Conclusion and Recommendations

This study has conducted a much-needed and novel investigation into the integration of AI code generation tools in summative assessments.

Learners have reported that utilising AI code generation tools has helped them greatly in looking at code from a different perspective and has therefore refined their code-review skills and heightened their awareness of diverse coding approaches.

Further insights from this study suggest that it is important to train students on the best practices of using such tools. This includes wording queries and detecting mistakes.

References

Biswas, Som (2023). "Role of ChatGPT in Computer Programming.: ChatGPT in Computer Programming." In: Mesopotamian Journal of Computer Sci-ence 2023, pp. 8–16.

Lau, Sam and Philip Guo (2023). "From" Ban it till we understand it" to" Re-sistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as Chat- GPT and GitHub Copilot". In: Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1, pp. 106–121.

Lim, Weng Marc et al. (2023). "Generative AI and the future of education: Ragnar¨ok or reformation? A paradoxical perspective from management educators". In: The International Journal of Management Education 21.2, p. 100790.

Sullivan, Miriam, Andrew Kelly, and Paul McLaughlan (2023). "ChatGPT in higher education: Considerations for academic integrity and student learning". In